

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA AN TOÀN THÔNG TIN



Môn Học: Thực Tập Cơ Sở

Báo Cáo Thực Hành Bài 15

Lập trình Client/Server để trao đổi thông tin

Giảng viên:	TS. Phạm Hoàng Duy
Sinh viên:	Hoàng Anh Khoa
Mã sinh viên:	B22DCAT164
Hệ đào tạo:	Đại học chính quy

Hà Nội, 4/2025

Mục Lục

1. Mục đích.....	2
2. Nội dung thực hành.....	2
2.1. Cơ sở lý thuyết: Tìm hiểu về các khái niệm liên quan đến lập trình socket với TCP.....	2
2.2. Các bước thực hiện	5
2.2.1. Chuẩn bị môi trường	5
2.2.2. Lập trình client và server với TCP socket.....	5
2.2.3. Trao đổi thông điệp giữa client và server và đảm bảo tính toàn vẹn của thông điệp khi trao đổi	8
3. Kết luận	12
4. Tài liệu tham khảo.....	12

Môn học Thực tập cơ sở

Bài 15: Lập trình client/server để trao đổi thông tin an toàn

1. Mục đích

Sinh viên hiểu về cơ chế client/server và có thể tự lập trình client/server dựa trên socket, sau đó thực hiện ca đặt giao thức đơn giản để trao đổi thông tin an toàn.

2. Nội dung thực hành

2.1. Cơ sở lý thuyết: Tìm hiểu về các khái niệm liên quan đến lập trình socket với TCP

a. Socket là gì?

Socket là điểm cuối end-point trong liên kết truyền thông hai chiều (two-way communication) biểu diễn kết nối giữa Client – Server. Các lớp Socket được ràng buộc với một cổng port (thể hiện là một con số cụ thể) để các tầng TCP (TCP Layer) có thể định danh ứng dụng mà dữ liệu sẽ được gửi tới.

Socket là giao diện lập trình ứng dụng mạng được dùng để truyền và nhận dữ liệu trên internet. Giữa hai chương trình chạy trên mạng cần có một liên kết giao tiếp hai chiều, hay còn gọi là two-way communication để kết nối 2 process trò chuyện với nhau. Điểm cuối (endpoint) của liên kết này được gọi là socket.

Một chức năng khác của socket là giúp các tầng **TCP** hoặc **TCP Layer** định danh ứng dụng mà dữ liệu sẽ được gửi tới thông qua sự ràng buộc với một cổng port (thể hiện là một con số cụ thể), từ đó tiến hành kết nối giữa client và server.

b. Tại sao người dùng lại cần dùng đến socket?

Người dùng cần đến socket vì nó là một cách tiêu biểu để thiết lập và quản lý kết nối mạng giữa các thiết bị và ứng dụng trên Internet. Dưới đây là một số lý do chính:

Giao tiếp mạng: Socket cho phép ứng dụng gửi và nhận dữ liệu qua mạng, cho phép giao tiếp giữa các thiết bị từ xa. Điều này là cần thiết trong nhiều ứng dụng như trò chơi trực tuyến, chat, truyền tệp, và nhiều ứng dụng mạng khác.

Tính linh hoạt: Socket hỗ trợ nhiều loại kết nối mạng khác nhau như TCP, UDP, và các giao thức khác, cung cấp sự linh hoạt trong cách ứng dụng giao tiếp với nhau.

Phân phối dữ liệu: Sử dụng socket, dữ liệu có thể được phân phối từ một nguồn tới nhiều đích hoặc ngược lại. Điều này cho phép triển khai các ứng dụng mạng phức tạp như streaming video, trò chơi trực tuyến, và các ứng dụng đa người dùng.

Tương thích đa nền tảng: Socket có sẵn trên nhiều nền tảng và hệ điều hành khác nhau, từ máy tính cá nhân đến thiết bị di động, giúp cho việc phát triển ứng dụng mạng trở nên dễ dàng và linh hoạt hơn.

Kiểm soát độ trễ: Sử dụng socket, người dùng có thể kiểm soát và tối ưu hóa độ trễ trong việc truyền và nhận dữ liệu qua mạng, điều này quan trọng đặc biệt trong các ứng dụng đòi hỏi độ trễ thấp như trò chơi trực tuyến và ứng dụng thời gian thực.

Tóm lại, socket là một công cụ quan trọng trong việc phát triển các ứng dụng mạng, cho phép giao tiếp hiệu quả và đa dạng giữa các thiết bị và ứng dụng trên Internet.

c. Điều kiện để Socket hoạt động?

Như đã đề cập trước đó, chức năng của **socket** là kết nối giữa client và server thông qua **TCP/IP** và **UDP** để truyền và nhận giữ liệu qua Internet. Giao diện lập trình ứng dụng mạng này chỉ có thể hoạt động khi đã có thông tin về thông số IP và số hiệu cổng của 2 ứng dụng cần trao đổi dữ liệu cho nhau.

2 ứng dụng cần truyền thông tin phải đáp ứng điều kiện sau thì socket mới có thể hoạt động:

- 2 ứng dụng có thể nằm cùng trên một máy hoặc 2 máy khác nhau
- Trong trường hợp 2 ứng dụng cùng nằm trên một máy, số hiệu cổng không được trùng nhau.

d. Phân loại Socket?

Socket có thể được phân loại chủ yếu dựa trên hai tiêu chí: giao thức và cách sử dụng.

Dưới đây là một số phân loại phổ biến của socket:

Theo giao thức:

- Socket TCP (Transmission Control Protocol): Sử dụng giao thức TCP để thiết lập kết nối đáng tin cậy, có kiểm soát lỗi và bảo đảm dữ liệu đến được đích một cách chính xác.
- Socket UDP (User Datagram Protocol): Sử dụng giao thức UDP, không đảm bảo tính toàn vẹn hoặc độ tin cậy, nhưng thích hợp cho các ứng dụng cần truyền dữ liệu nhanh mà không cần quá nhiều kiểm soát.

Theo cách sử dụng:

- Socket Server: Là socket được sử dụng để lắng nghe và chấp nhận kết nối từ các client khác.
- Socket Client: Là socket được sử dụng để thiết lập kết nối và truyền dữ liệu đến một socket server.

Mỗi loại socket có mục đích và ứng dụng khác nhau trong lập trình mạng. Sử dụng đúng loại socket sẽ giúp đảm bảo tính ổn định và hiệu suất của ứng dụng mạng.

e. Lập trình socket với TCP/IP

Các tiến trình mà muốn truyền thông với nhau thì sẽ gửi thông điệp thông qua các socket. Socket là cánh cửa của tiến trình ứng dụng và giao thức tầng transport (ở đây là TCP).

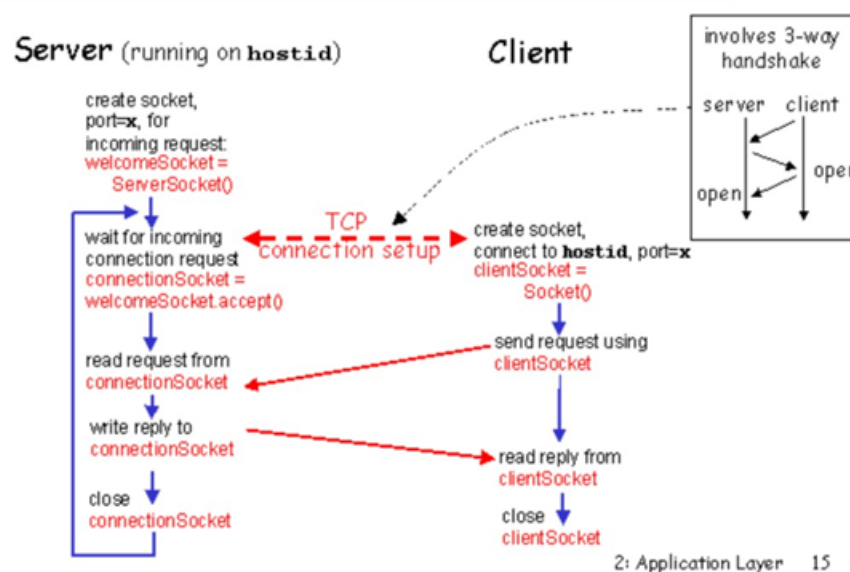
Để có thể tương tác với nhau và máy chủ có thể nhận liên lạc từ máy khách thì máy chủ phải luôn sẵn sàng. Điều này có 2 nghĩa, thứ nhất, giống như trường hợp của UDP, một tiến trình của máy chủ phải được chạy trước khi máy khách khởi tạo liên lạc đến. Thứ hai, chương trình chủ phải tạo ra 1 socket để sẵn sàng chấp nhận kết nối từ tiến trình khách.

Khi tiến trình chủ đã chạy, lúc này tiến trình khách sẽ tạo ra 1 socket TCP để có thể kết nối đến máy chủ. Trong khi máy khách đang tạo TCP socket, nó sẽ đặc tả địa chỉ IP, số cổng của tiến trình chủ.

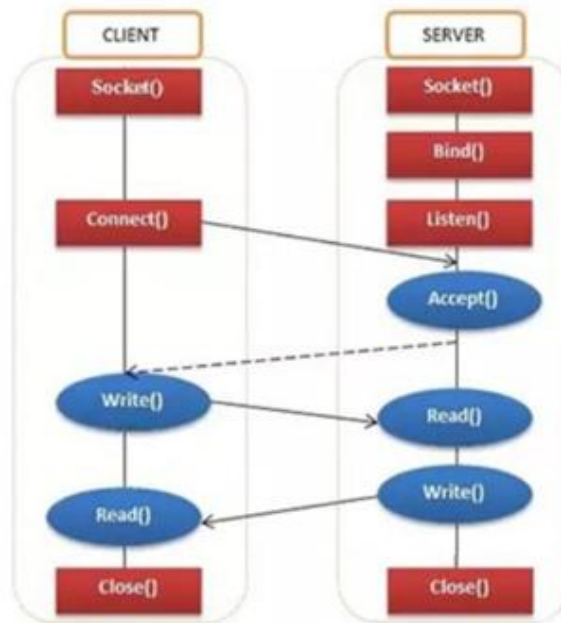
Khi socket của tiến trình khách vừa được tạo, TCP trên máy khách sẽ tiến hành thực hiện quá trình bắt tay 3 bước và thiết lập kết nối TCP tới máy chủ.

Trong quá trình bắt tay 3 bước, khi tiến trình chủ nhận thấy tiến trình khách, nó sẽ tự tạo ra 1 socket mới chỉ dành riêng cho tiến trình khách đó. Khi được máy khách gõ cửa, chương trình kích hoạt với phương thức `accept()`. Cuối quá trình bắt tay 3 bước, một kết nối TCP tồn tại giữa socket của máy khách và socket của máy chủ.

Client/server socket interaction: TCP



Mô tả quá trình:



1. Trước tiên chúng ta sẽ tạo ra một máy chủ bằng cách mở một socket - Socket()
2. Sau đó chúng ta sẽ liên kết nó với một host hoặc một máy và một port - Bind()
3. Tiếp theo server sẽ bắt đầu lắng nghe trên port đó - Listen()
4. Yêu cầu kết nối từ client được gửi tới server - Connect()
5. Server sẽ accept yêu cầu từ client và sau đó kết nối được thiết lập - Accept()
6. Bây giờ cả hai đều có thể gửi và nhận tin tại thời điểm đó - Read() / Write()
7. Và cuối cùng khi hoàn thành chúng có thể đóng kết nối - Close()

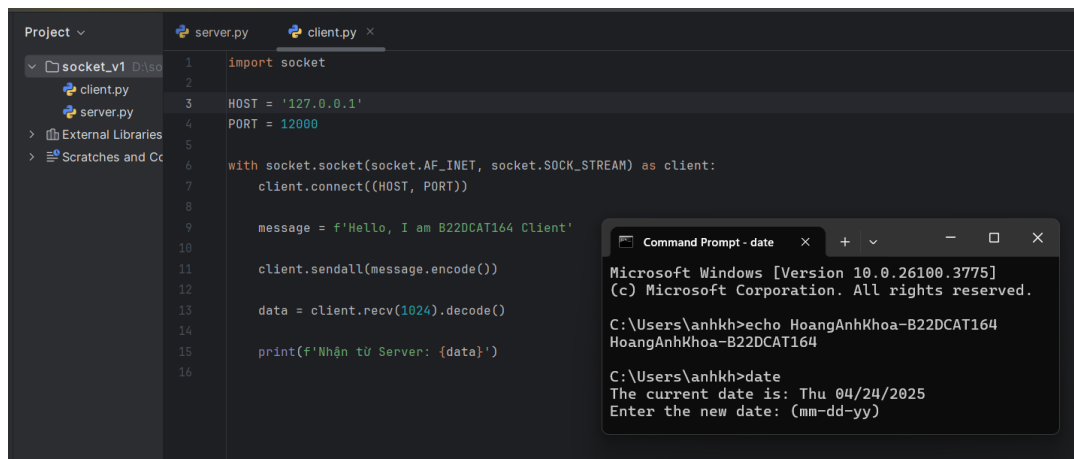
2.2. Các bước thực hiện

2.2.1. Chuẩn bị môi trường

- Môi trường Python hoặc Java để chạy được ứng dụng client/server đã lập trình.
- Phần mềm Wireshark

2.2.2. Lập trình client và server với TCP socket

- Lập trình client



The screenshot shows a VS Code editor with a project named 'socket_v1'. The 'client.py' file is open, displaying the following code:

```
1 import socket
2
3 HOST = '127.0.0.1'
4 PORT = 12000
5
6 with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client:
7     client.connect((HOST, PORT))
8
9     message = f'Hello, I am B22DCAT164 Client'
10
11     client.sendall(message.encode())
12
13     data = client.recv(1024).decode()
14
15     print(f'Nhận từ Server: {data}')
```

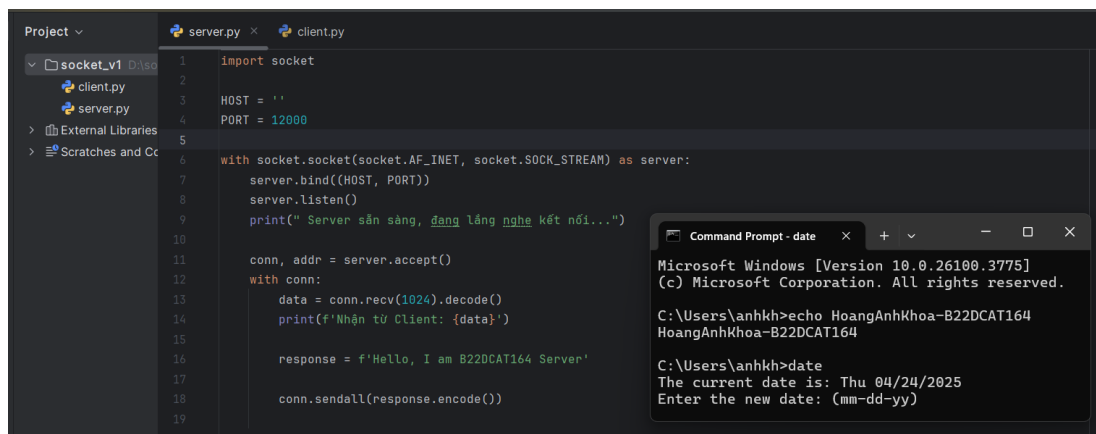
A terminal window titled 'Command Prompt - date' is open, showing the following output:

```
Microsoft Windows [Version 10.0.26100.3775]
(c) Microsoft Corporation. All rights reserved.

C:\Users\anhkh>echo HoangAnhKhoa-B22DCAT164
HoangAnhKhoa-B22DCAT164

C:\Users\anhkh>date
The current date is: Thu 04/24/2025
Enter the new date: (mm-dd-yy)
```

- Lập trình server



The screenshot shows a VS Code editor with a project named 'socket_v1'. The 'server.py' file is open, displaying the following code:

```
1 import socket
2
3 HOST = ''
4 PORT = 12000
5
6 with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
7     server.bind((HOST, PORT))
8     server.listen()
9     print(" Server sẵn sàng, đang lắng nghe kết nối...")
10
11     conn, addr = server.accept()
12     with conn:
13         data = conn.recv(1024).decode()
14         print(f'Nhận từ Client: {data}')

A terminal window titled 'Command Prompt - date' is open, showing the following output:



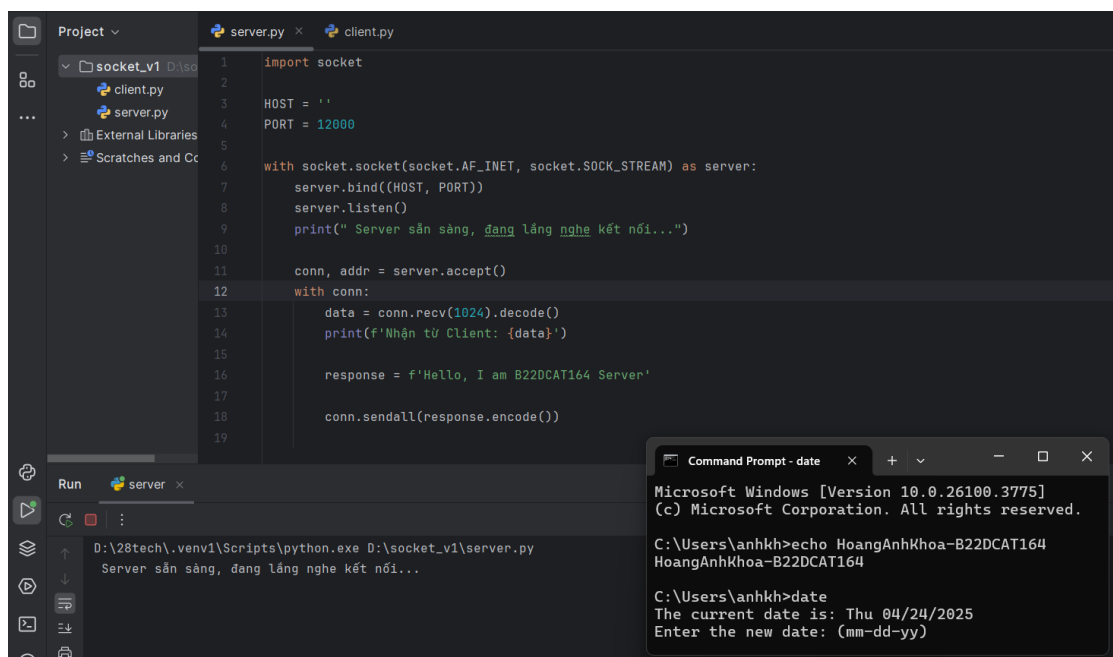
```
Microsoft Windows [Version 10.0.26100.3775]
(c) Microsoft Corporation. All rights reserved.

C:\Users\anhkh>echo HoangAnhKhoa-B22DCAT164
HoangAnhKhoa-B22DCAT164

C:\Users\anhkh>date
The current date is: Thu 04/24/2025
Enter the new date: (mm-dd-yy)
```


```

- Chạy server sau đó chạy client



The screenshot shows a VS Code editor with a project named 'socket_v1'. The 'server.py' file is open, displaying the following code:

```
1 import socket
2
3 HOST = ''
4 PORT = 12000
5
6 with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
7     server.bind((HOST, PORT))
8     server.listen()
9     print(" Server sẵn sàng, đang lắng nghe kết nối...")
10
11     conn, addr = server.accept()
12     with conn:
13         data = conn.recv(1024).decode()
14         print(f'Nhận từ Client: {data}')

The 'client.py' file is also open, displaying the following code:



```
1 import socket
2
3 HOST = '127.0.0.1'
4 PORT = 12000
5
6 with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client:
7 client.connect((HOST, PORT))
8
9 message = f'Hello, I am B22DCAT164 Client'
10
11 client.sendall(message.encode())
12
13 data = client.recv(1024).decode()
14
15 print(f'Nhận từ Server: {data}')
```



A terminal window titled 'Command Prompt - date' is open, showing the following output:



```
Microsoft Windows [Version 10.0.26100.3775]
(c) Microsoft Corporation. All rights reserved.

C:\Users\anhkh>echo HoangAnhKhoa-B22DCAT164
HoangAnhKhoa-B22DCAT164

C:\Users\anhkh>date
The current date is: Thu 04/24/2025
Enter the new date: (mm-dd-yy)
```


```

- Client gửi thông điệp cá nhân hóa cho server: “Hello, I am <mã sinh viên> client.”

The screenshot shows a VS Code editor with a project named 'socket_v1'. The 'client.py' file is open, displaying the following code:

```

1 import socket
2
3 HOST = '127.0.0.1'
4 PORT = 12000
5
6 with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client:
7     client.connect((HOST, PORT))
8
9     message = f'Hello, I am B22DCAT164 Client'
10
11     client.sendall(message.encode())
12
13 data = client.recv(1024).decode()
14
15 print(f'Nhận từ Server: {data}')
16

```

The terminal window shows the output of running the client script:

```

D:\28tech\.venv1\Scripts\python.exe D:\socket_v1\client.py
Nhận từ Server: Hello, I am B22DCAT164 Server
Process finished with exit code 0

```

A separate Command Prompt window is also visible, showing the user's input and the system's response:

```

C:\Users\anhkh>echo HoangAnhKhoa-B22DCAT164
HoangAnhKhoa-B22DCAT164

C:\Users\anhkh>date
The current date is: Thu 04/24/2025
Enter the new date: (mm-dd-yy) |

```

- Server nhận được hiển thị thông điệp nhận được và gửi lại client thông điệp: server gửi lại “Hello, I am <mã sinh viên> server”

The screenshot shows a VS Code editor with a project named 'socket_v1'. The 'server.py' file is open, displaying the following code:

```

1 import socket
2
3 HOST = ''
4 PORT = 12000
5
6 with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
7     server.bind((HOST, PORT))
8     server.listen()
9     print(" Server sẵn sàng, đang lắng nghe kết nối...")
10
11 conn, addr = server.accept()
12 with conn:
13     data = conn.recv(1024).decode()
14     print(f'Nhận từ Client: {data}')
15
16     response = f'Hello, I am B22DCAT164'
17
18     conn.sendall(response.encode())
19

```

The terminal window shows the output of running the server script:

```

D:\28tech\.venv1\Scripts\python.exe D:\socket_v1\server.py
Server sẵn sàng, đang lắng nghe kết nối...
Nhận từ Client: Hello, I am B22DCAT164 Client
Process finished with exit code 0

```

A separate Command Prompt window is also visible, showing the user's input and the system's response:

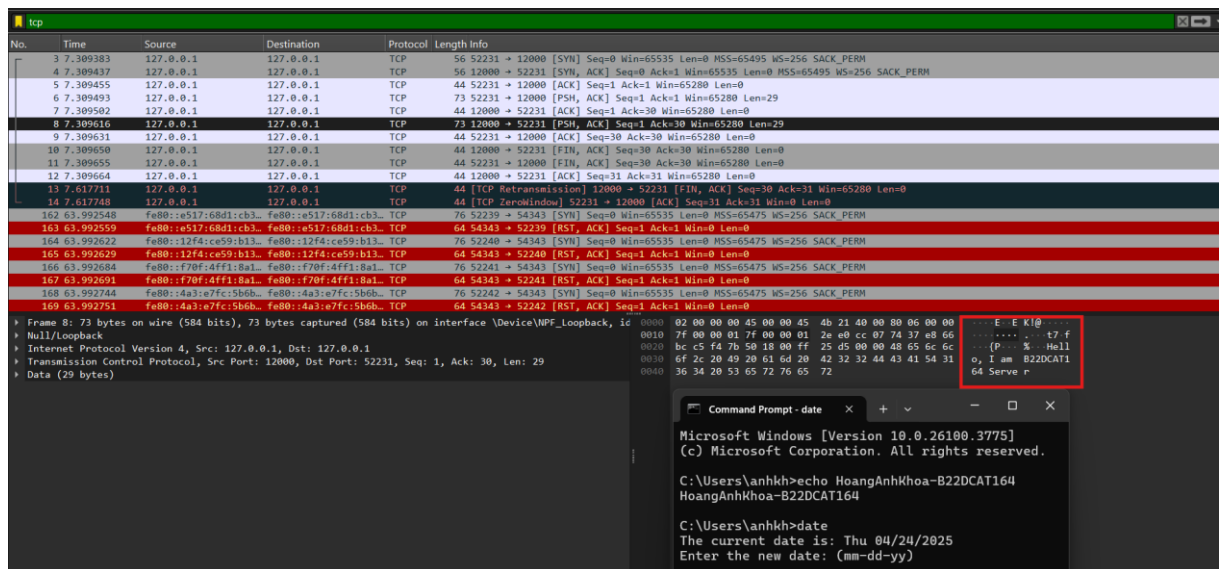
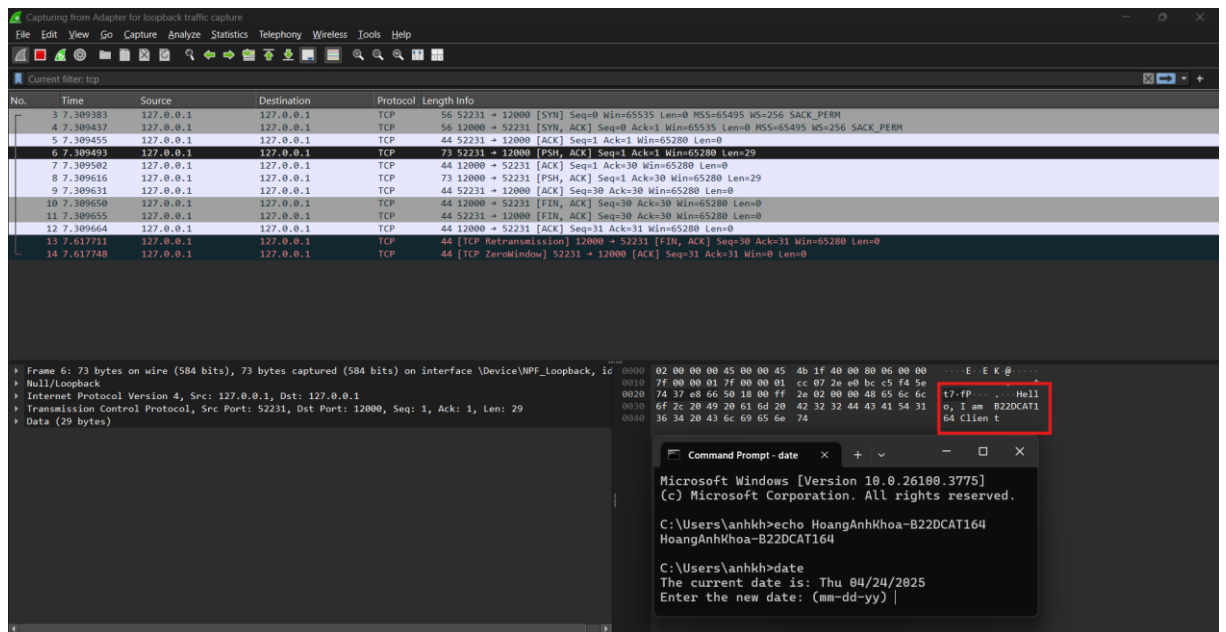
```

C:\Users\anhkh>echo HoangAnhKhoa-B22DCAT164
HoangAnhKhoa-B22DCAT164

C:\Users\anhkh>date
The current date is: Thu 04/24/2025
Enter the new date: (mm-dd-yy)

```

- Sử dụng Wireshark để bắt các thông tin đã gửi từ client đến server và ngược lại



2.2.3. Trao đổi thông điệp giữa client và server và đảm bảo tính toàn vẹn của thông điệp khi trao đổi

- Từ client và server, sửa đổi để sao cho: khi gửi thông điệp sẽ gửi kèm theo giá trị băm của (thông điệp + key) để phía bên kia kiểm tra xác minh tính toàn vẹn. Hai bên có thể thống nhất một giá trị key trước đó.

The screenshot shows a code editor with a project named 'socket_v1'. The 'server.py' file is open, displaying the following code:

```

1 import socket
2 import hashlib
3
4 HOST = ''
5 PORT = 12000
6 KEY = 'B22DCAT164'
7
8 def compute_hash(msg, key): 1 usage
9     return hashlib.sha256((msg + key).encode()).hexdigest()
10
11 with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
12     server.bind((HOST, PORT))
13     server.listen()
14     conn, addr = server.accept()
15     with conn:
16         raw = conn.recv(4096).decode()
17         message, rcv_hash = raw.split(sep='||', maxsplit=1)
18
19         expected = compute_hash(message, KEY)
20
21         print(f'[Server] Received hash: {rcv_hash}')
22         print(f'[Server] Computed hash: {expected}')
23
24         if rcv_hash == expected:
25             print(f'✅ Integral message: {message}')
26             reply = 'Integral message'
27             conn.sendall(f'{reply}||{expected}'.encode())
28         else:
29             print(f'❌ The received message has lost its integrity.')
30             conn.sendall('The received message has lost its integrity.'.encode())
31

```

A terminal window titled 'Command Prompt - date' is open, showing the following commands and output:

```

C:\Users\anhkh>echo HoangAnhKhoa-B22DCAT164
HoangAnhKhoa-B22DCAT164

C:\Users\anhkh>date
The current date is: Thu 04/24/2025
Enter the new date: (mm-dd-yy)

```

Code server

The screenshot shows a code editor with a project named 'socket_v1'. The 'client.py' file is open, displaying the following code:

```

1 import socket
2 import hashlib
3
4 HOST = '127.0.0.1'
5 PORT = 12000
6 KEY = 'B22DCAT164'
7
8 def compute_hash(msg, key): 1 usage
9     return hashlib.sha256((msg + key).encode()).hexdigest()
10
11 with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client:
12     client.connect((HOST, PORT))
13
14     message = input('Enter message: ')
15     mac = compute_hash(message, KEY)
16
17     print(f'[Client] Computed hash: {mac}')
18
19     client.sendall(f'{message}||{mac}'.encode())
20
21     data = client.recv(4096).decode()
22
23     if data.startswith('The received message'):
24         print(data)
25     else:
26         reply, srv_mac = data.split(sep='||', maxsplit=1)
27         print(f'[Client] Server reply hash: {srv_mac}')
28         print(f'✅ From server: {reply}')
29

```

A terminal window titled 'Command Prompt - date' is open, showing the following commands and output:

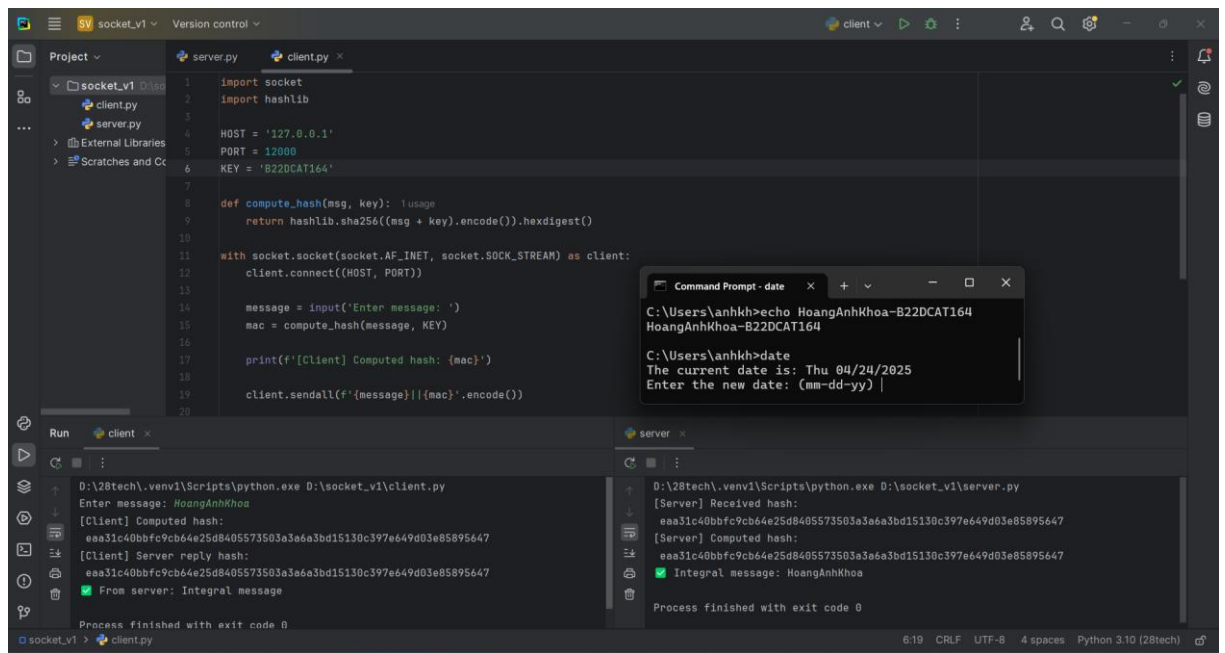
```

C:\Users\anhkh>echo HoangAnhKhoa-B22DCAT164
HoangAnhKhoa-B22DCAT164

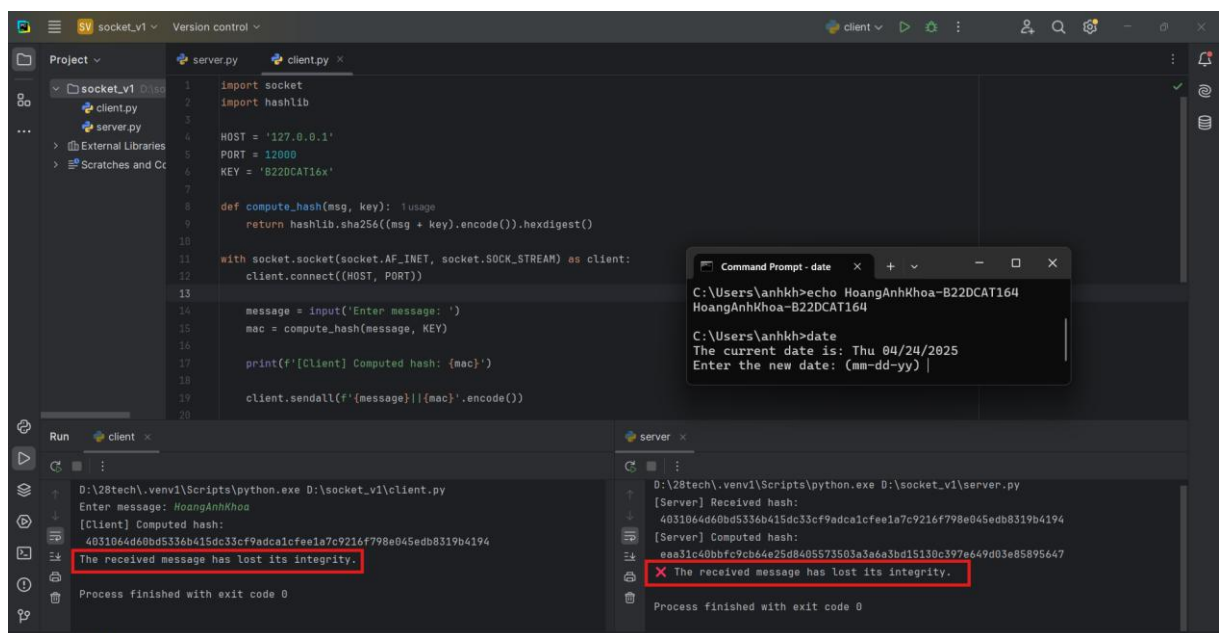
C:\Users\anhkh>date
The current date is: Thu 04/24/2025
Enter the new date: (mm-dd-yy)

```

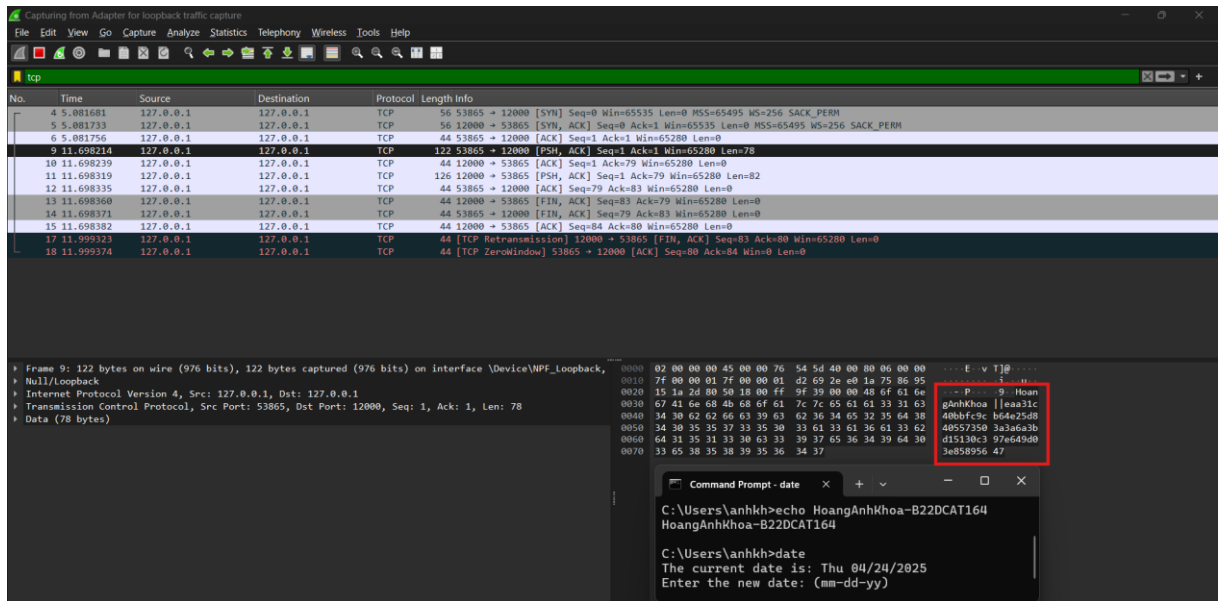
Code Client



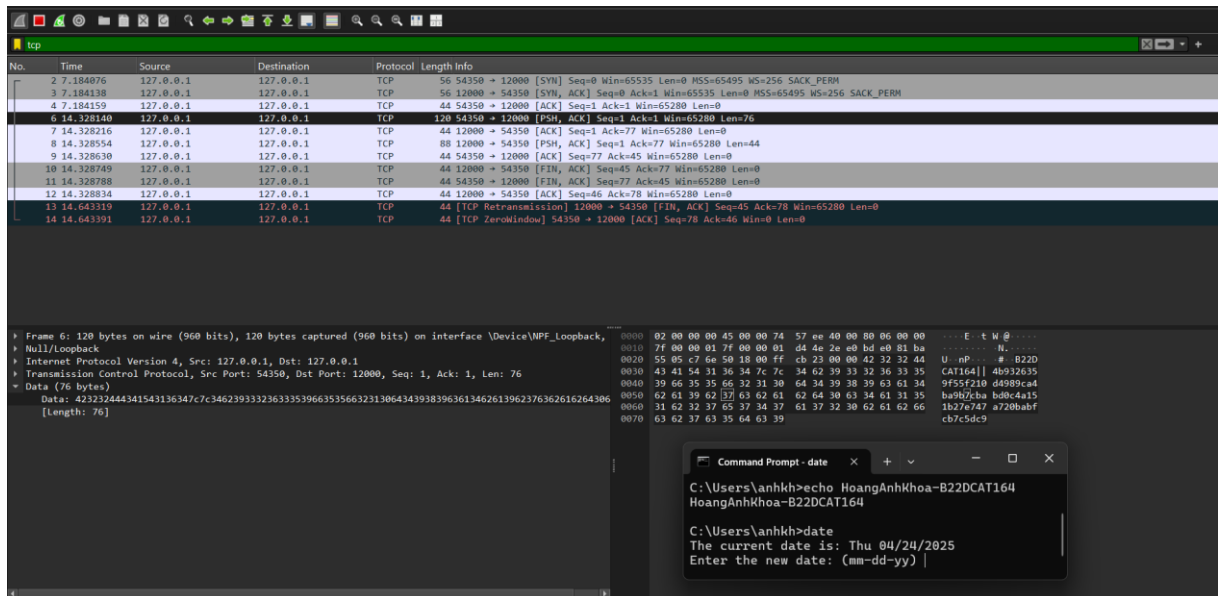
- Thay đổi giá trị key tại client và thực hiện gửi lại, nếu không đáp ứng tính toàn vẹn cần thông báo: “The received message has lost its integrity.”

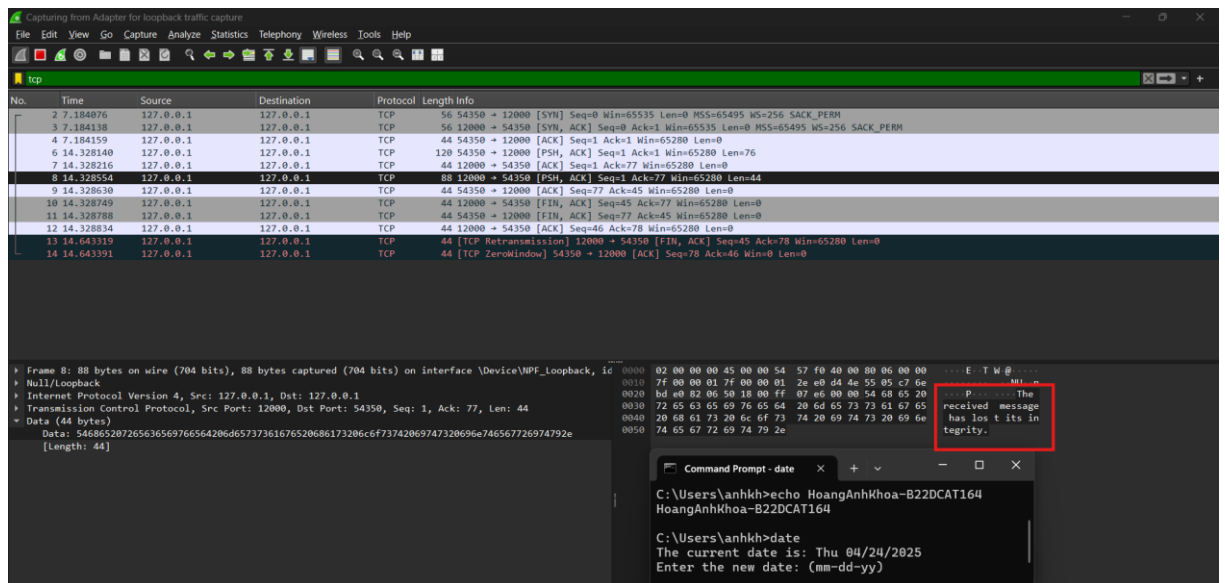


- Bắt được các bản tin trao đổi giữa client và server trong Wireshark trường hợp key giống nhau



- Bắt được các bản tin trao đổi giữa client và server trong Wireshark trong trường hợp key bị thay đổi





Server phản hồi Client: dữ liệu đã mất mát

3. Kết luận

- Lập trình và chạy thành công TCP Socket Client/Server
- Trao đổi thông điệp giữa client và server và đảm bảo tính toàn vẹn của thông điệp khi trao đổi
- Bắt được các bản tin trao đổi giữa client và server trong Wireshark

4. Tài liệu tham khảo

- [1]. Tham khảo tài liệu: Chapter 2: Application Layer V8.1 (9/2020) tại địa chỉ http://gaia.cs.umass.edu/kurose_ross/ppt.php (chú ý ví dụ từ trang 105).
- [2]. [Hướng dẫn code Socket, xem ở đây](#)